



Protecting the App!

Protecting Applications!

- **What is the threat model?**
 - who are our adversaries?
- **How to deal with changing threats?**
 - new threats will appear over time?
 - how can we react to changing threats?
- How to deal with vulnerabilities?
 - how can we track them?
 - how can we mitigate them?
 - how to reduce the cost of applications
- developing the application by reusing code?
- no customization for security?
- How to keep control of data / app
 - that is managed by an external entity?
 - that is implemented by an external entity?
- ...

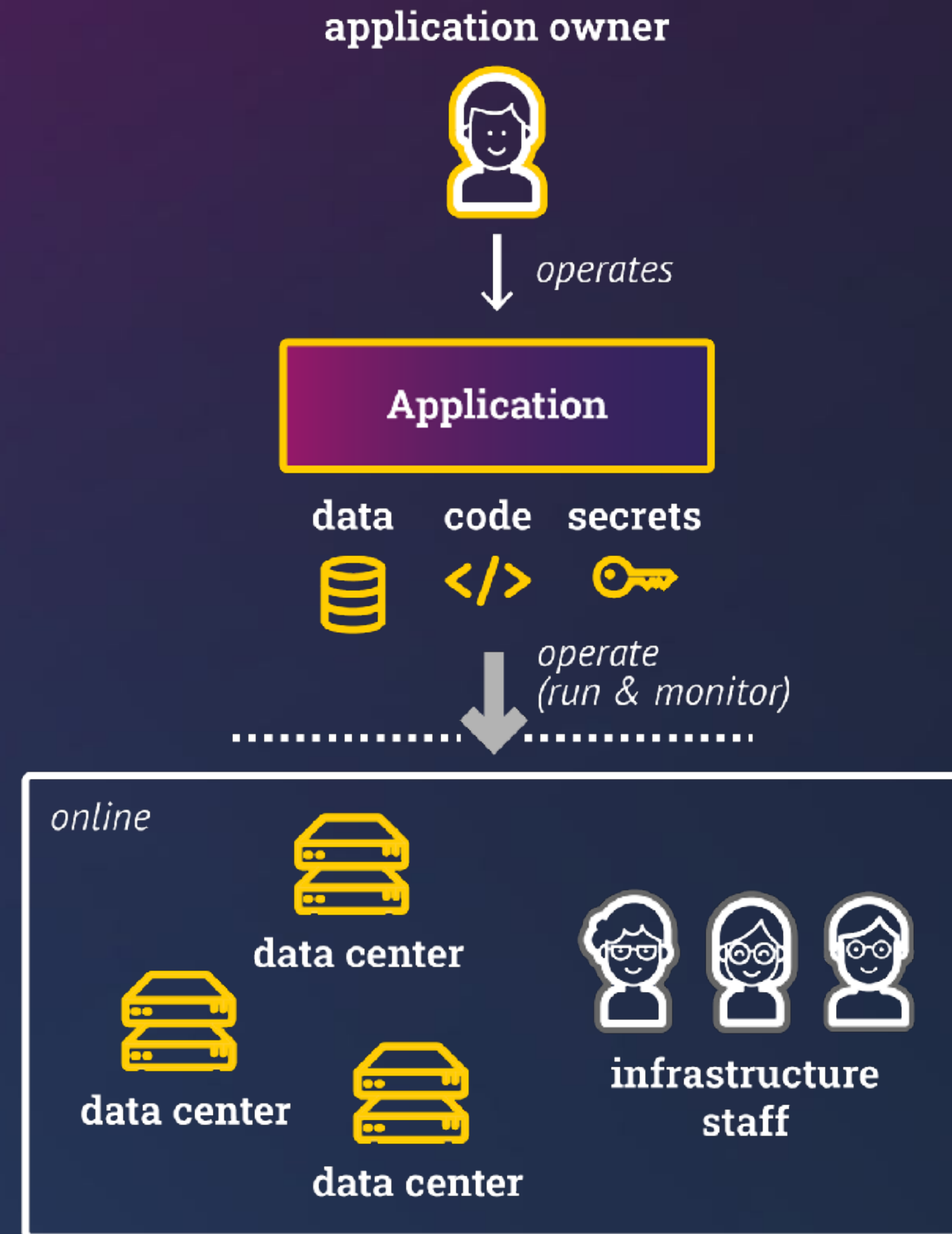
Focus: Deployment, Secrets, and ConfigMaps?



Objective: Isolate Infrastructure Team

Role: **infrastructure staff**

- everybody who can log onto the server or touch the hardware (e.g., Kubernetes admin)



Objective: Isolate Infrastructure Team

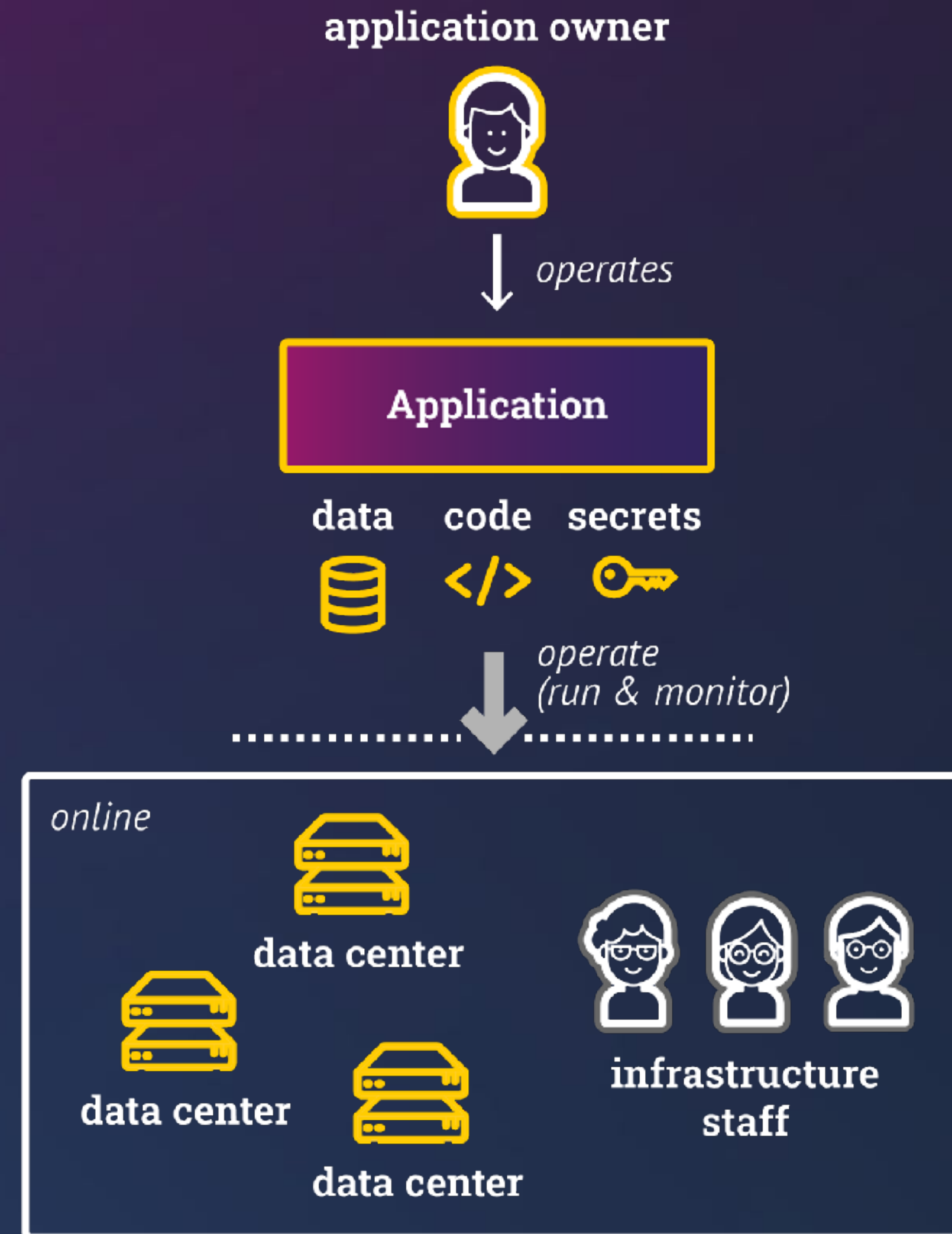
Role: infrastructure staff

- everybody who can log onto the server or touch the hardware (e.g., Kubernetes admin)



Objectives: infrastructure staff

- no access to **data, secrets, configurations**
- cannot** change data, secrets, configurations



Case 1:

Access to KubeAPI

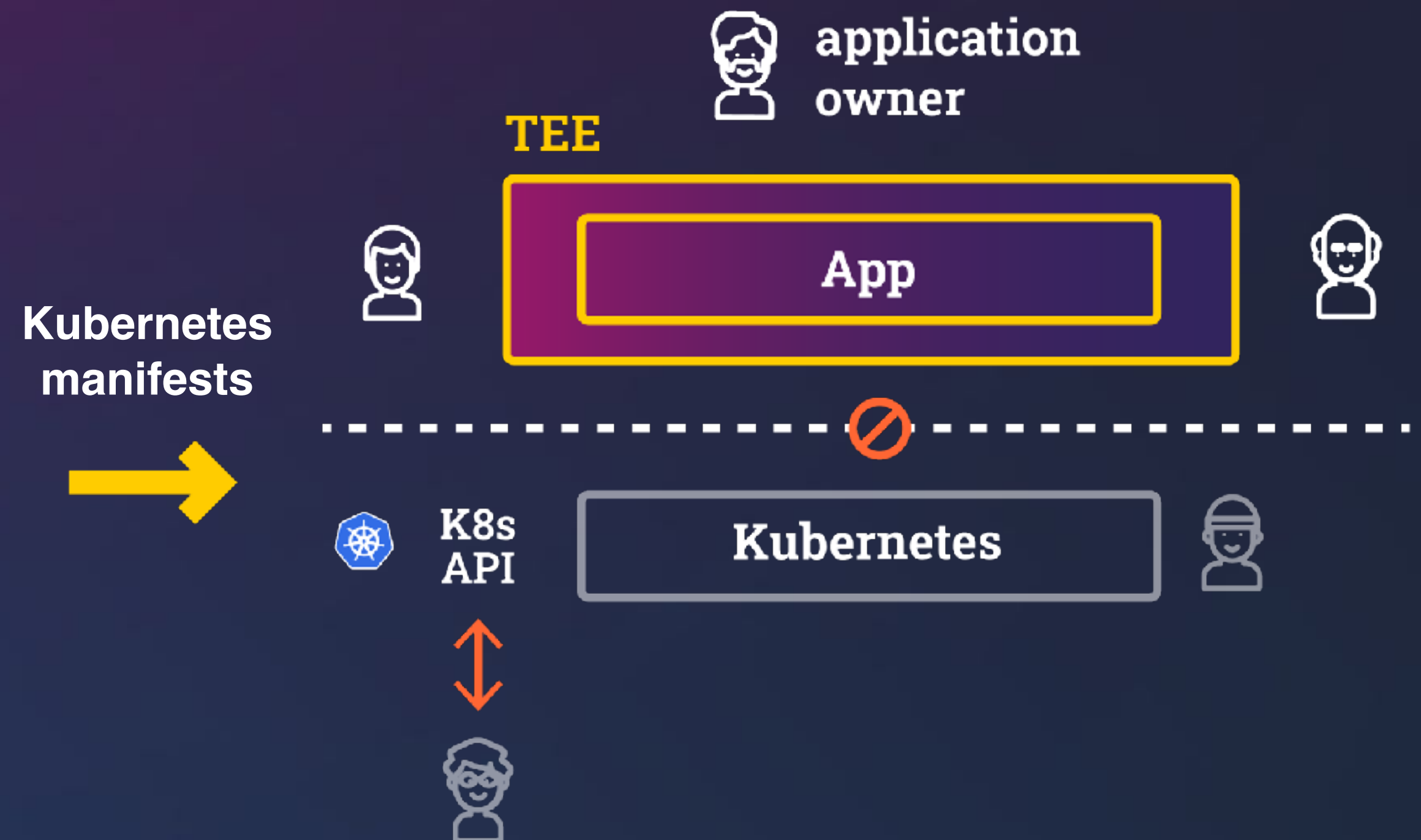
- We can show the secrets using script
 - `./scripts/decode_secret.sh`
- Secrets are `base64` encoded, i.e.,
 - easy to decode

```
🔑 Secret key: secrets
Encoded (base64):
cGFzc3dvcmQ6IHMzY3IzdApteV9kY19lbmNyeXB0aW9uX2tleTogd
Decoded:
password: s3cr3t
my_db_encryption_key: top_secret_key
```


Threat Model: Cannot Trust Managed Kubernetes

② How to protect **Secrets** and **ConfigMaps** and ... from Kubernetes?

- Use of managed Kubernetes
 - Kubernetes **admins** can read these in the **clear**!
- Kubernetes **admins** can modify the manifests, e.g.,
 - spawning extra containers inside pod to access files
 - read/modify **secrets** and **configMaps**

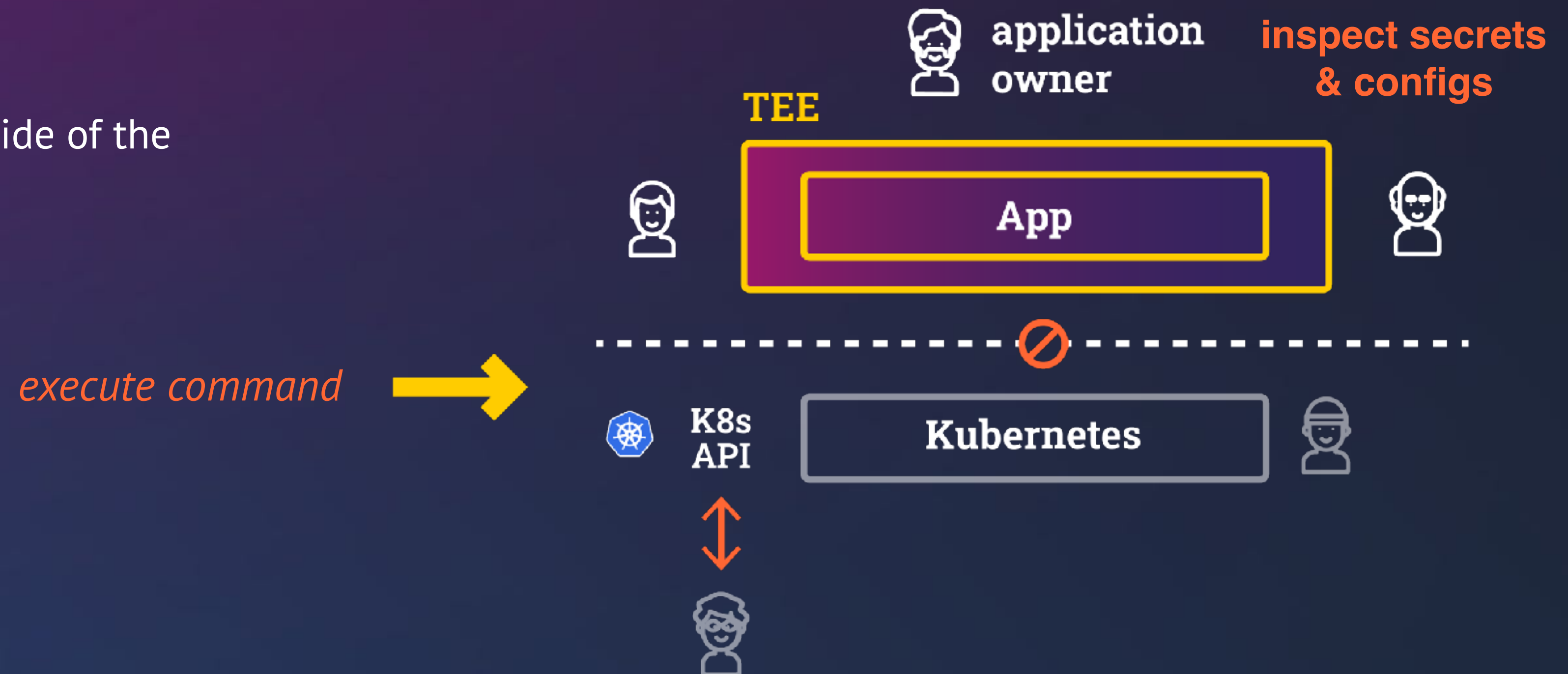


⚡ **Problem:**
Neither K8s nor K8s admins can be trusted!

Threat Model: Log Into Container

② How to protect **Secrets** and **ConfigMaps** and ... from Kubernetes?

- Secrets and ConfigMaps are visible inside of the container



⚡ **Problem:**
K8s nor K8s admins can be trusted!

Demo: Inspecting Secrets

We can read the secrets and configurations by

- executing a shell inside the container

```
ubuntu@apple-1:java-args-env-file$ kubectl get pods
NAME                                READY    STATUS    RESTARTS    AGE
java-cli-env-reader-554d78485c-vf2z5 1/1      Running   0           59m
...
ubuntu@apple-1:java-args-env-file$ k exec -it java-cli-env-reader-554d78485c-vf2z5 -- bash
root@java-cli-env-reader-554d78485c-vf2z5:/app# ls /config/
configs.yaml  secrets  secrets~
root@java-cli-env-reader-554d78485c-vf2z5:/app# cat /config/configs.yaml
key1: value1
key2: value2
root@java-cli-env-reader-554d78485c-vf2z5:/app# cat /config/secrets
password: s3cr3t
my_db_encryption_key: top_secret_key
```


Other Attack Vectors

- If adversary controls the **host** (OS, Hypervisor)
 - adversary can enter the containers and read the secrets of containers running inside of VMs
- If adversary controls the injection of ssh public keys into the **(C)VM**
 - adversary can inject different public ssh key into the (C)VM
 - adversary can enter the **VM** via ssh and hence, the **containers**
- ... quite a few ways to get to the data, secrets, and configuration